



Máquinas de Estado

INFO II

Profesor: Ing. Federico Weber

TEMAS

- ¿Qué son?
- ¿Por qué usarlas?
- Máquinas de Mealy y Moore
- Implementación de una máquina de estados en C
- Ejemplo 1: Contador de cantidad de 1 impares
- Ejemplo 2 y 3: Detector ABCD (+ contador)
- EFSM
- Ejemplo 4: Tratamiento de protocolo de comunicaciones
- TFSM
- Ejemplo 5: Anti-rebote
- Ejercicios

¿Qué son las Máquinas de Estado?

- FSM: Finite State Machine
- Es un método para modelizar sistemas. Estos sistemas tienen una cantidad finita (y fija) de estados (de ahí el “Finite State”) y cuentan con entradas y salidas discretas. El comportamiento del sistema va a depender del estado en el que se encuentre.

¿Qué son las Máquinas de Estado?

Otros nombres / formalismos:

- Autómata finito (teoría de lenguajes)
- LTS (labelled transition system)
- IOA (input – output Automata)

Tipos / variantes:

- por la salida → Moore / Mealy
- por guardar datos → EFSM (extended)
- por el tiempo → TFSM (timed)
- por jerarquía → Statechart;
- por determinismo → DFSM / NFSM

¿Qué son las Máquinas de Estado?

El modelo del comportamiento del sistema denota su evolución mediante:

- Estados (comportamiento estático)
- Transiciones (evolución entre estados)
- Excitaciones/entradas (que condicionan las transiciones)
- Acciones/salidas (asociadas a las transiciones)

Se pueden representar de dos formas: como gráficos (STD: *State Transition Diagram*) o como tabla (STT: *State Transition Table*)

¿Qué son las Máquinas de Estado?

- Si bien nosotros vamos a usar las FSM como una abstracción para programar, se puede usar para modelizar cualquier sistema que cumpla los requisitos.
- Vemos un ejemplo:

¿Qué son las Máquinas de Estado?



¿Qué son las Máquinas de Estado?



¿Qué son las Máquinas de Estado?



¿Qué son las Máquinas de Estado?



Observamos que, **aunque la entrada sea siempre la misma**, el comportamiento de la “**maquina**” es diferente ya que el “**estado del sistema**” es otro.
Ingresando siempre las mismas entradas el sistema no tiene que llegar a los mismos estados.

¿Qué son las Máquinas de Estado?



Observamos que, **aunque la entrada sea siempre la misma**, el comportamiento de la “**maquina**” es diferente ya que el “**estado del sistema**” es otro.
Ingresando siempre las mismas entradas el sistema no tiene que llegar a los mismos estados.

Representemos este simple sistema utilizando un diagrama de maquinas de estado (**STD**)

¿Qué son las Máquinas de Estado?



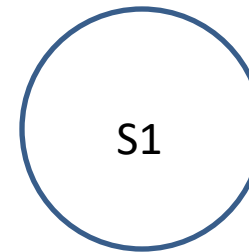
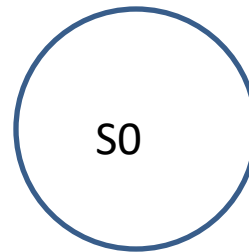
Hay dos estados posibles:

S0: retraída

S1: extendida

Entradas:

In: Press Button



¿Qué son las Máquinas de Estado?



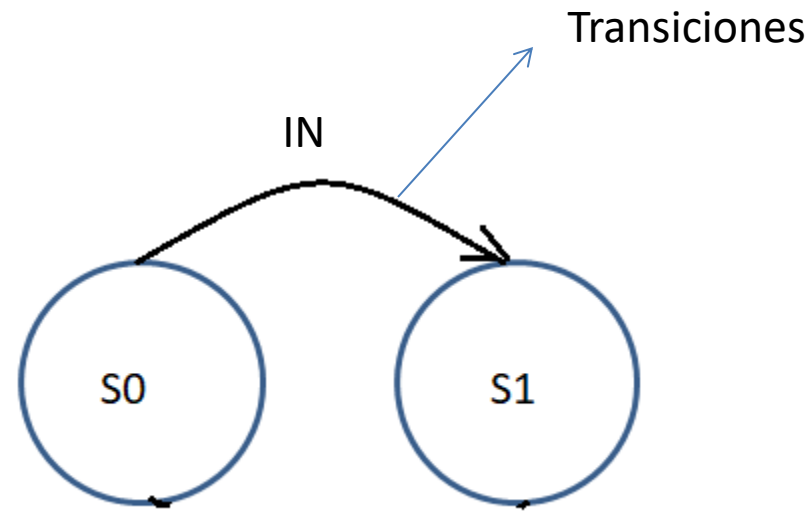
Hay dos estados posibles:

S0: retraída

S1: extendida

Entradas:

In: Press Button



¿Qué son las Máquinas de Estado?



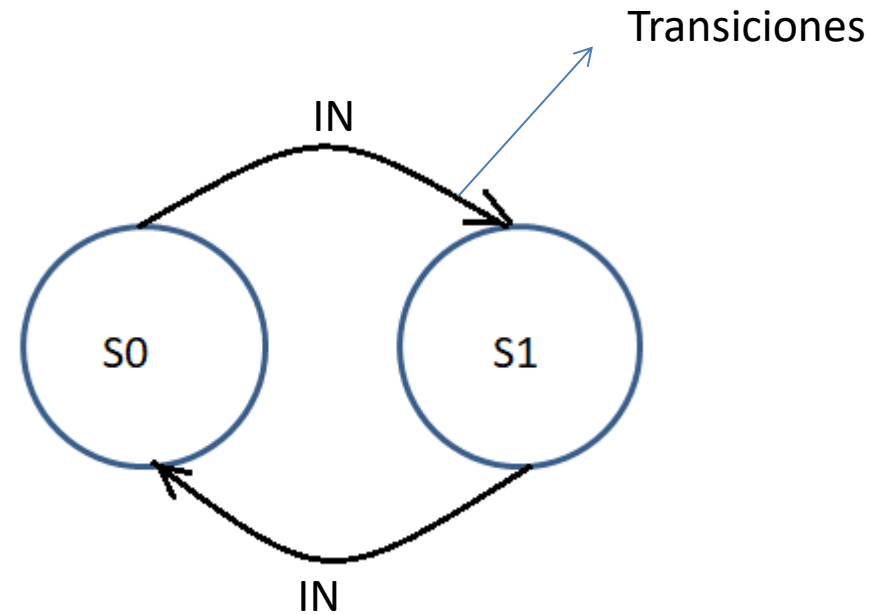
Hay dos estados posibles:

S0: retraída

S1: extendida

Entradas:

In: Press Button



¿Qué son las Maquinas de Estado?



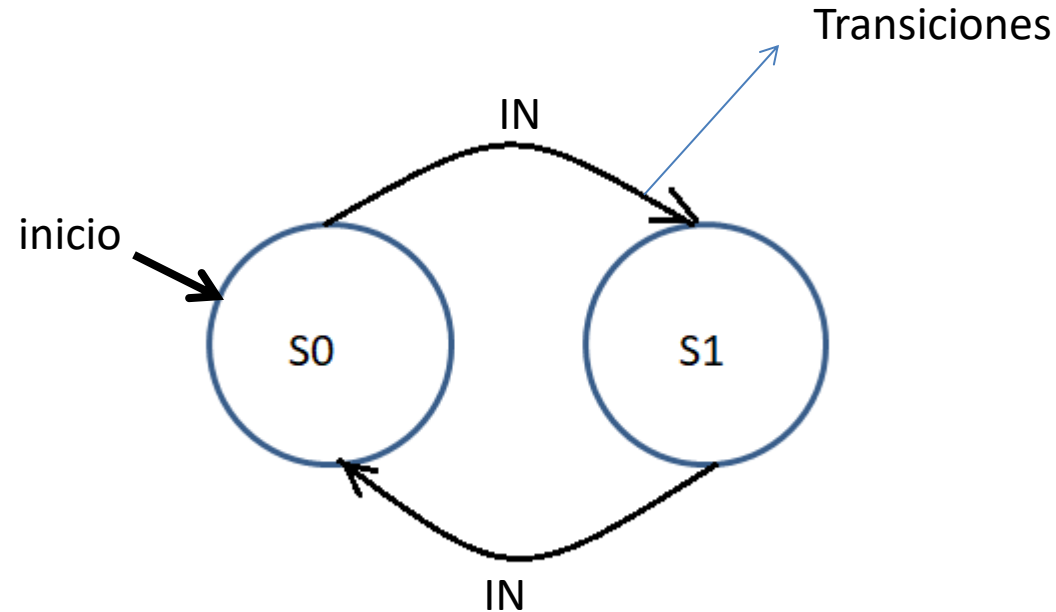
Hay dos estados posibles:

S0: retraída

S1: extendida

Entradas:

In: Press Button



¿Qué son las Máquinas de Estado?



Hay dos estados posibles:

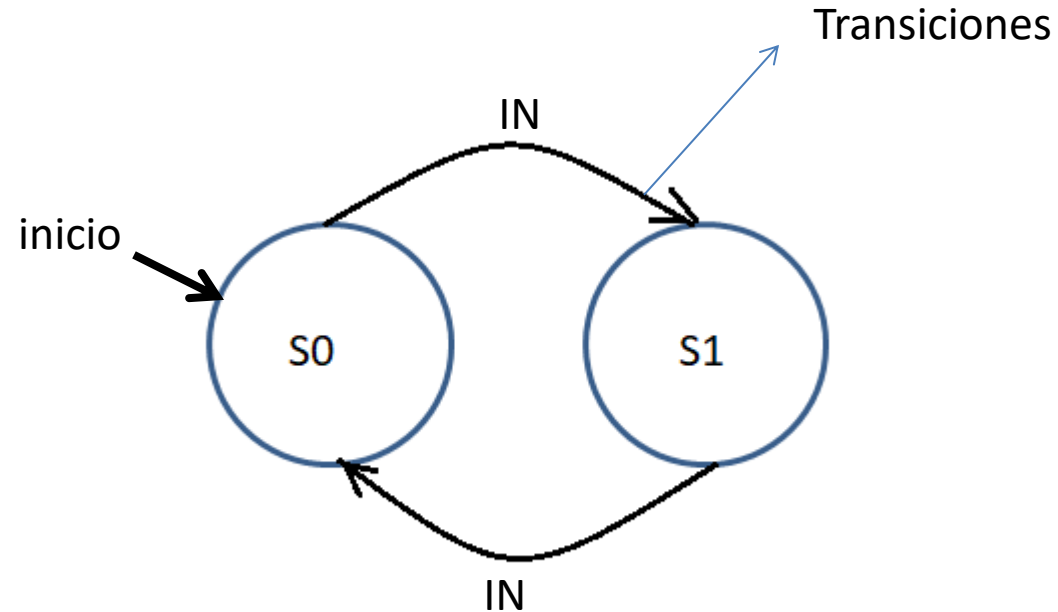
S0: retraída

S1: extendida

Entradas:

In: Press Button

¿Las
salidas?



TEMAS

- ¿Qué son?
- ¿Por qué usarlas?
- Máquinas de Mealy y Moore
- Implementación de una máquina de estados en C
- Ejemplo 1: Contador de cantidad de 1 impares
- Ejemplo 2 y 3: Detector ABCD (+ contador)
- EFSM
- Ejemplo 4: Tratamiento de protocolo de comunicaciones
- TFSM
- Ejemplo 5: Anti-rebote
- Ejercicios

¿Por qué usar maquinas de estado?

- Para ver esto pongamos como ejemplo un cajero automático:



¿Por qué usar maquinas de estado?

- Para ver esto pongamos como ejemplo un cajero automático:



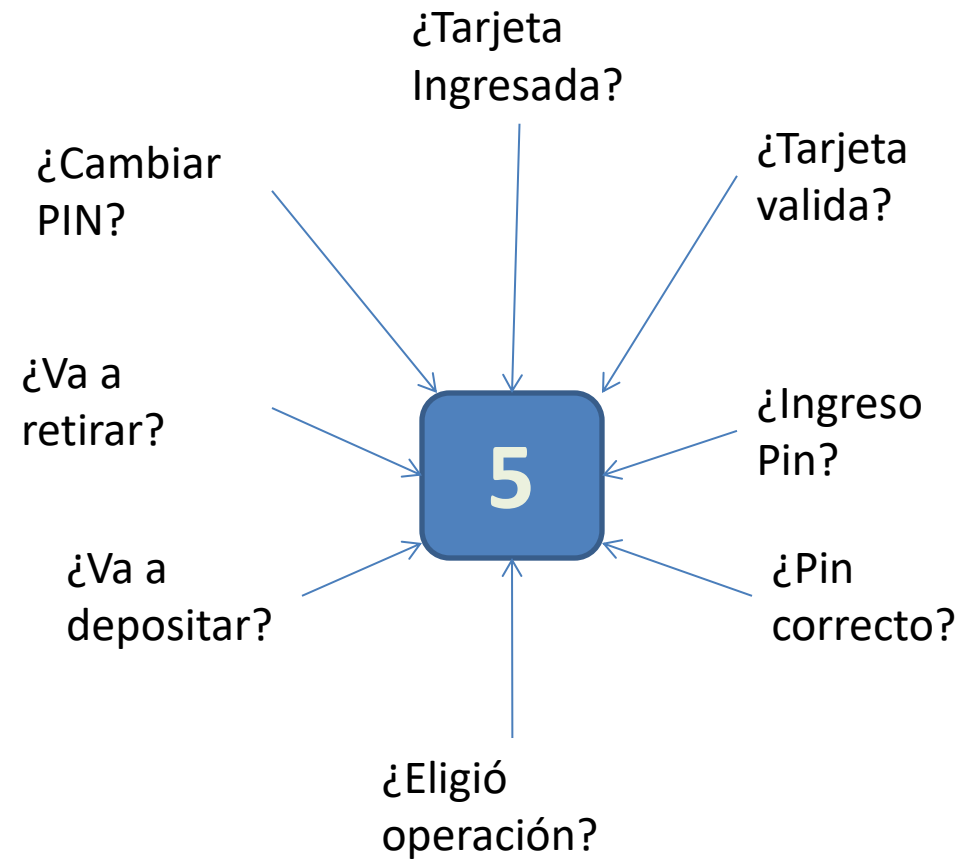
Un usuario se acerca y presiona el botón “5”

¿Cómo debe responder el programa?

¿Por qué usar maquinas de estado?



¿Por qué usar maquinas de estado?



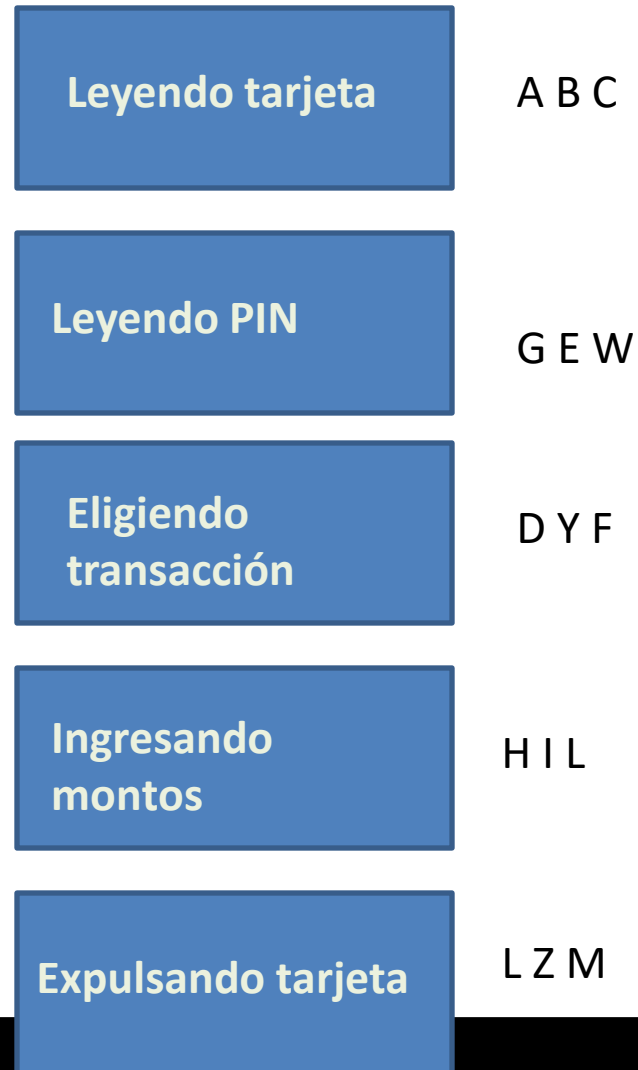
¿Por qué usar maquinas de estado?



```
if (card_swiped){  
    if (pin_entered){  
        if (transaction_selected){  
            if (cash_withdrawal) {  
                if (amount_selected){  
                    confirm_amount (a,b,c);  
                }  
            }  
            else {  
                continue_entry (d,e,f);  
            }  
        }  
        else if (depositing_cash){  
            if (amount_selected){  
                confirm_amount (g,h,i);  
            }  
            else {  
                continue_entry (k,k,l);  
            }  
        }  
    }  
}
```

VARIABLES: A , B , C , D , E , F , G , H , I , J , K , L etc

¿Por qué usar maquinas de estado?



¿Por qué usar maquinas de estado?

- Brinda una rigurosa descripción del comportamiento del sistema.
- Soluciones muy sofisticadas y sencillas.
- Se puede utilizar soluciones compuestas por la operación en paralelo de múltiples maquinas de estado.
- Código mas simple, eficiente y preciso. Mas fácil de depurar, modificar, expandir y mejor organizado.
- Facilitan el análisis, el diseño y la implementación.

¿Por qué usar maquinas de estado?

- Interpretar una maquina de estados es mucho mas conciso que leer la descripción detallada del funcionamiento de una maquina.
- Es un punto de entrada de muy alto nivel para ciertos proyectos. Incluso hay software (por ej. **itemis CREATE** -ex YAKINDU Statechart Tools-) que ingresando la maquina de estados te devuelve el código.

TEMAS

- ¿Qué son?
- ¿Por qué usarlas?
- Máquinas de Mealy y Moore
- Implementación de una máquina de estados en C
- Ejemplo 1: Contador de cantidad de 1 impares
- Ejemplo 2 y 3: Detector ABCD (+ contador)
- EFSM
- Ejemplo 4: Tratamiento de protocolo de comunicaciones
- TFSM
- Ejemplo 5: Anti-rebote
- Ejercicios

Maquinas de Moore y Mealy

- Ambas sirven para exactamente lo mismo. La diferencia radica en como manejan las salidas.
- **Maquina de Moore:**
 $\text{next_state} = f(\text{estado actual}, \text{inputs})$
 $\text{output} = g(\text{estado actual})$

Output en el estado

- **Maquina de Mealy:**
 $\text{next_state} = f(\text{estado actual}, \text{inputs})$
 $\text{output} = g(\text{estado actual}, \text{input})$

Output en la transición

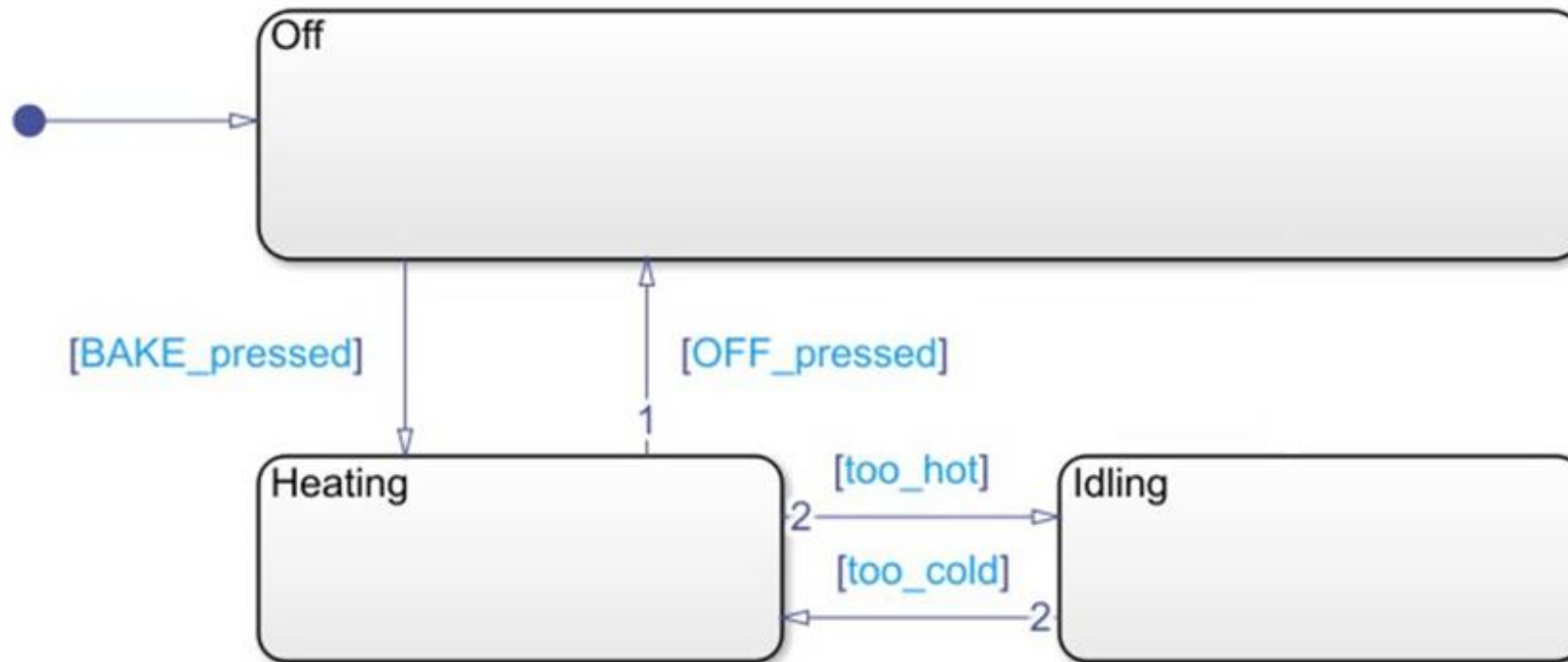
Maquinas de Moore y Mealy

Veamos las diferencias con un ejemplo:

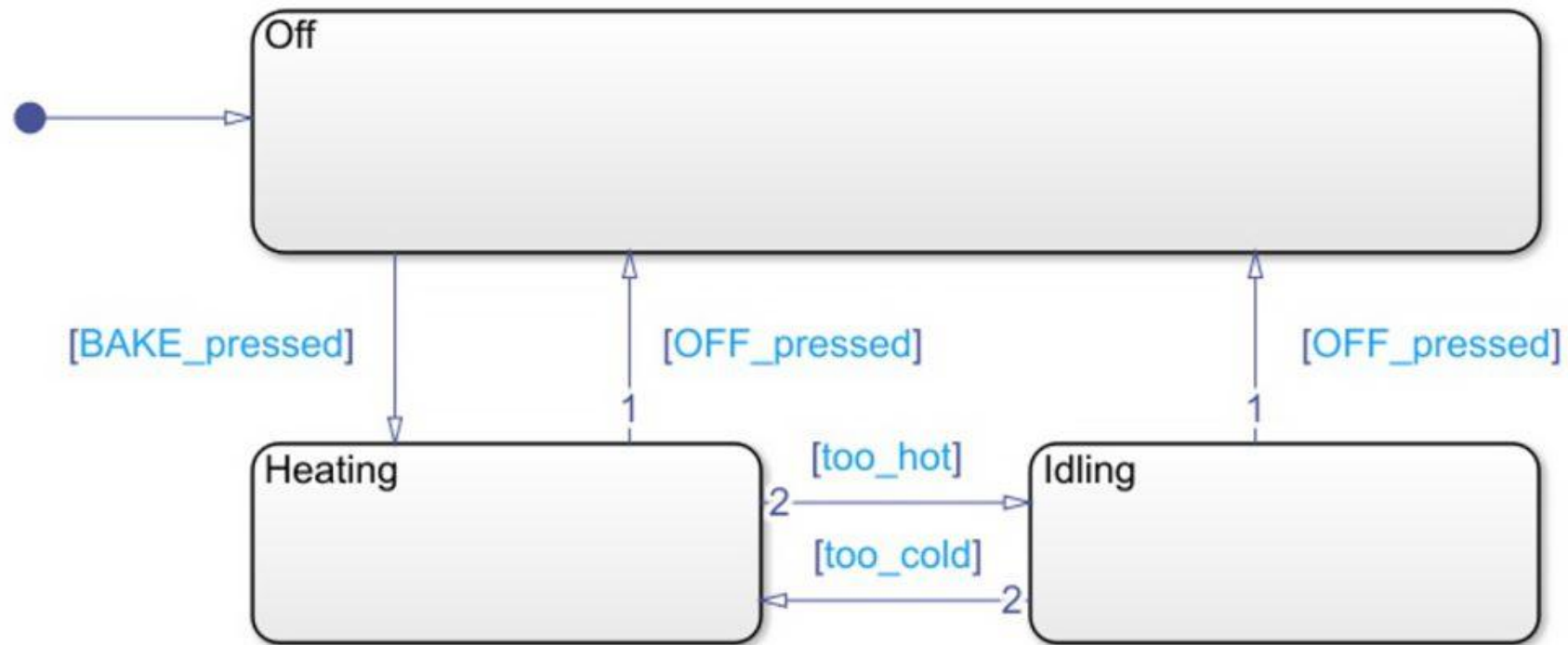


Máquinas de Moore y Mealy

¿Encuentran algún problema?



Maquinas de Moore y Mealy

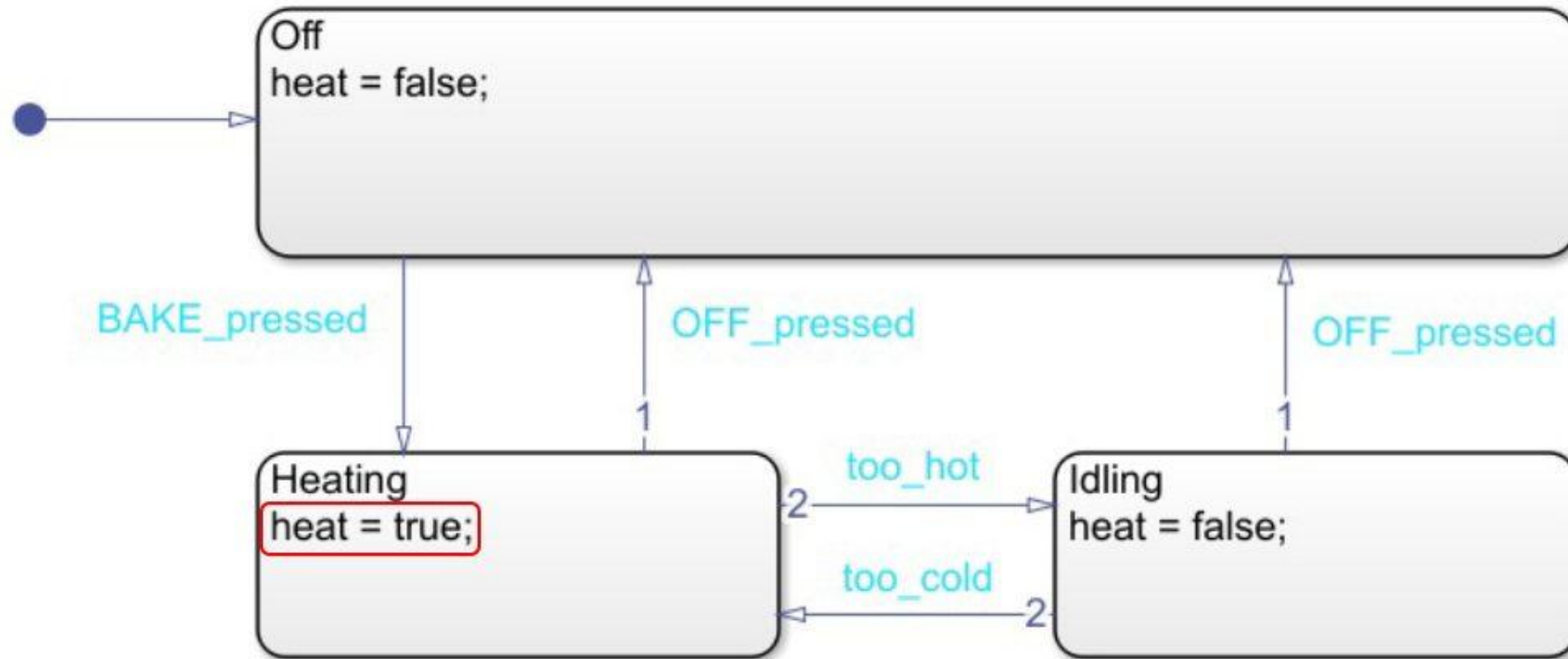


Maquinas de Moore

$\text{next_state} = f(\text{estado actual}, \text{inputs})$

$\text{output} = g(\text{estado actual})$

Output en el estado

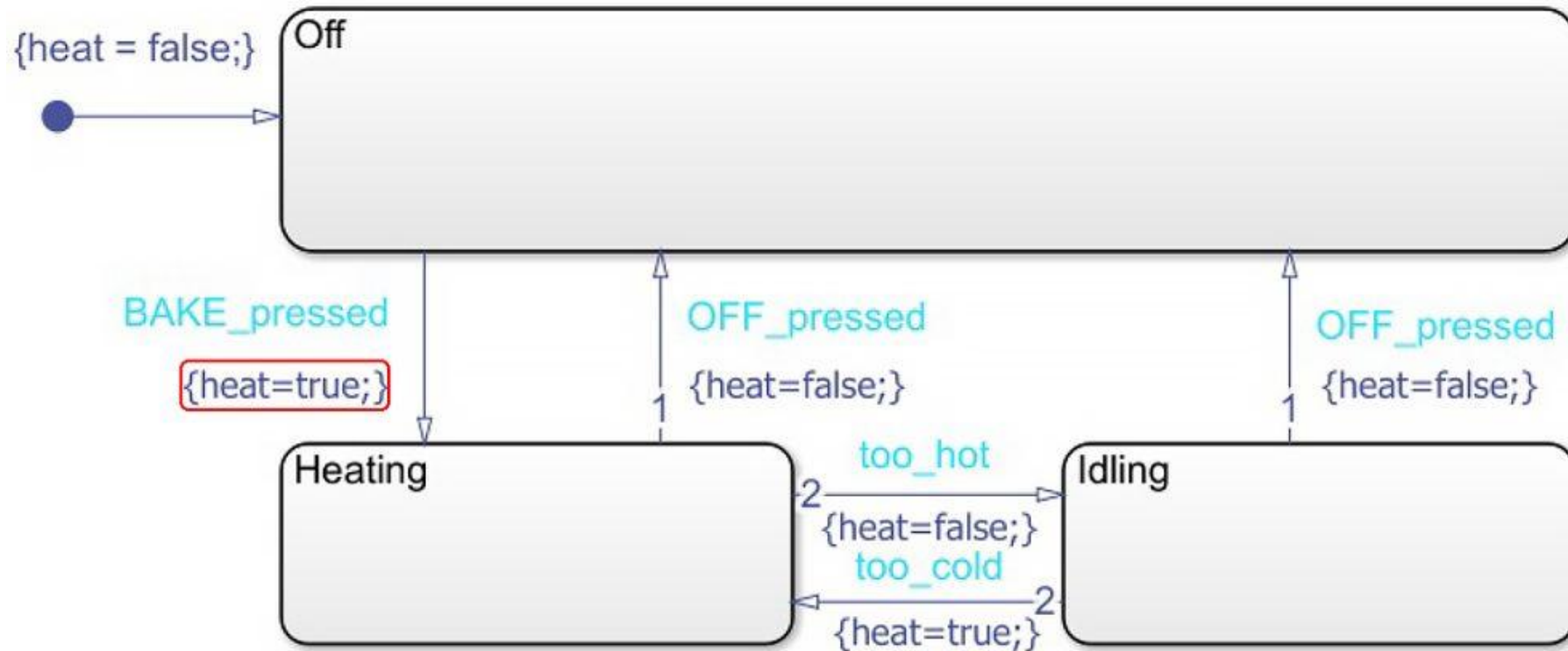


Maquinas de Mealy

$\text{next_state} = f(\text{estado actual}, \text{inputs})$

$\text{output} = g(\text{estado actual}, \text{input})$

Output en la transición



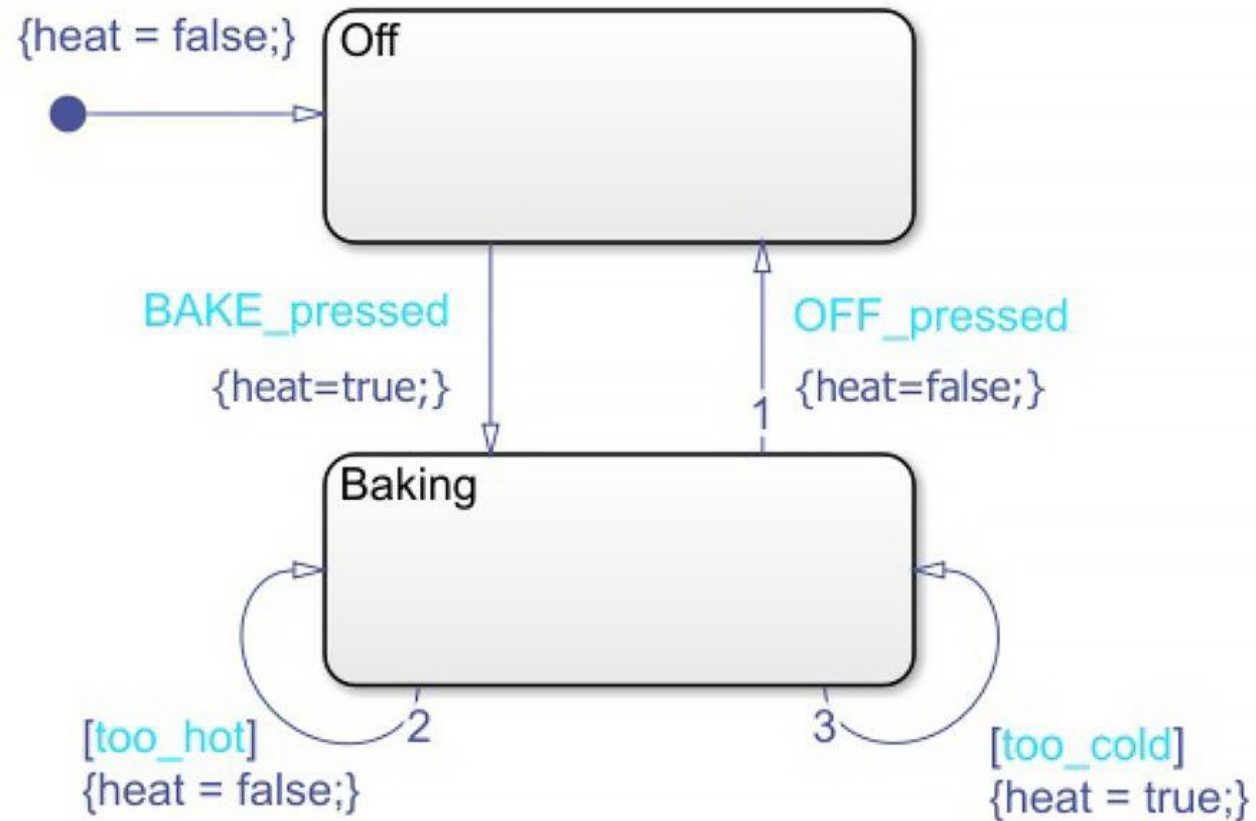
Ahora las salidas SI dependen de las entradas. Usando Mealy podemos simplificar la maquina.

Maquinas de Mealy

$\text{next_state} = f(\text{estado actual}, \text{inputs})$

$\text{output} = g(\text{estado actual}, \text{input})$

Output en la transición



Mealy and Moore

- Moore, Edward F (1956). “Gedanken-Experiments on Sequential Machines.” Automata Studies, Annals of Mathematical Studies (Princeton, N.J.: Princeton University Press)(34): 129 – 153
- Mealy, George H. (September 1955). “A Method for Synthesizing Sequential Circuits.” Bell Systems Technical Journal 34: 1045-1079

TEMAS

- ¿Qué son?
- ¿Por qué usarlas?
- Máquinas de Mealy y Moore
- Implementación de una máquina de estados en C
- Ejemplo 1: Contador de cantidad de 1 impares
- Ejemplo 2 y 3: Detector ABCD (+ contador)
- EFSM
- Ejemplo 4: Tratamiento de protocolo de comunicaciones
- TFSM
- Ejemplo 5: Anti-rebote
- Ejercicios

Implementación de una maquina de estados en C

- La iremos viendo en tres formas, de la más simple a la más potente
 - 1) switch case y enumeraciones (primer clase)
 - 2) punteros a función (segunda clase)
 - 3) tabla de transición (más adelante...)

Implementación de una maquina de estados en C – caso **switch case** y enumeraciones

- Realizar el diagrama de estados
- Realizar una definicion **typedef** de un **enum** para definir cada estado
- Una variable del tipo **enum**, de paso anterior para definir cada estado.
- Crear un ciclo **while** infinito para ejecutar cada estado.
- Se debe definir el estado inicial de la maquina.
- Un **switch** para ejecutar el estado siguiente.
- Dentro de cada **case** el código a ejecutar del caso y a evaluación para determinar el siguiente caso.

Enumeraciones y typedef

INFO II
Profesor: Ing. Federico
Weber

Enumeraciones

```
enum DiaSemana{  
    lunes,  
    martes,  
    miercoles,  
    jueves,  
    viernes,  
    sabado,  
    domingo,  
} ;
```

```
int main() {  
    enum DiaSemana dia = miercoles;  
  
    if (dia == miercoles)  
        printf("Hoy es miercoles. Mitad de semana.\n");  
    else  
        printf("No es miercoles.\n");  
    getch();  
  
    return 0;  
}
```

- Es un tipo de dato compuesto que puede ser definido por el usuario.

Enumeraciones

- En C++ existen las enumeraciones sin ámbito y con ámbito. Por ahora solo veremos sin ámbito que son las únicas que existen en C.
- Vamos al IDE. Mostrar error al querer repetir enumeraciones.

Typedef

Se utiliza para crear alias de tipos en C y C++

```
1  #include <stdio.h>
2
3  // Definición de un alias para un tipo de datos
4  typedef unsigned long ulong;
5
6  int main() {
7      ulong a = 100;
8      printf("Valor de a: %lu\n", a);
9      return 0;
10 }
11
```

Typedef

Se utiliza para crear alias de tipos en C y C++

```
1  #include <stdio.h>
2
3  // Definición de un alias para un tipo de datos
4  typedef struct {
5      int id;
6      float nota;
7  } Alumnos;
8
9  int main() {
10     Alumnos alumno ;
11     alumno.id = 1;
12     alumno.nota = 3.0;
13     return 0;
14 }
15
```

Typedef

Se utiliza para crear alias de tipos en C y C++

```
1  #include <stdio.h>
2  #include <conio.h>
3
4  // Definición de una enumeración para los días de la semana
5  typedef enum {
6      lunes,
7      martes,
8      miercoles,
9      jueves,
10     viernes,
11     sabado,
12     domingo,
13 } DiaSemana;
14
15 // Función principal
16 int main() {
17     DiaSemana dia = miercoles; // Asignar un día de la semana
```

Implementación de una maquina de estados en C – caso switch case y enumeraciones

```
1  typedef enum {  
2      STATE_0,  
3      STATE_1,  
4      STATE_2,  
5      //  
6      //  
7      STATE_N  
8  } STATES;  
9  
10  
11  int main(void){  
12  
13      STATES NextState;  
14  
15      NextState = STATE_0 ;  
16  
17      while ("condición de salida") {  
18          switch (NextState) {  
19              case STATE_0:  
20                  //Code to run when STATE_0  
21                  if ( X == Y)  
22                      NextState = STATE_1;  
23                  else  
24                      NextState = STATE_N;  
25                  break;  
26  
27              case STATE_1:  
28                  //  
29  
30                  break;  
31  
32              default:  
33                  // When the selected State has not defnied show an error...  
34                  break;  
35          }  
36  
37      }  
38  }
```

Ejemplo 1

- Realizar un programa que permita ingresar por teclado cualquier secuencia de 0 y 1. En pantalla se debe mostrar si la cantidad de 1 ingresados es PAR o IMPAR. Utilizar máquinas de estado finito.

Ejemplo 1

- Realizar un programa que permita ingresar por teclado cualquier secuencia de 0 y 1. En pantalla se debe mostrar si la cantidad de 1 ingresados es PAR o IMPAR. Utilizar máquinas de estado finito.



Ejemplo 1

- Realizar un programa que permita ingresar por teclado cualquier secuencia de 0 y 1. En pantalla se debe mostrar si la cantidad de 1 ingresados es PAR o IMPAR. Utilizar maquinas de estado finito.



Ejemplo 1

- Realizar un programa que permita ingresar por teclado cualquier secuencia de 0 y 1. En pantalla se debe mostrar si la cantidad de 1 ingresados es PAR o IMPAR. Utilizar maquinas de estado finito.



Ejemplo 1

- Realizar un programa que permita ingresar por teclado cualquier secuencia de 0 y 1. En pantalla se debe mostrar si la cantidad de 1 ingresados es PAR o IMPAR. Utilizar maquinas de estado finito.



Ejemplo 1

- Realizar un programa que permita ingresar por teclado cualquier secuencia de 0 y 1. En pantalla se debe mostrar si la cantidad de 1 ingresados es PAR o IMPAR. Utilizar maquinas de estado finito.



Ejemplo 1

- Realizar un programa que permita ingresar por teclado cualquier secuencia de 0 y 1. En pantalla se debe mostrar si la cantidad de 1 ingresados es PAR o IMPAR. Utilizar maquinas de estado finito.



Identificamos los estados:

- PAR
- IMPAR

Ejemplo 1

- Realizar un programa que permita ingresar por teclado cualquier secuencia de 0 y 1. En pantalla se debe mostrar si la cantidad de 1 ingresados es PAR o IMPAR. Utilizar maquinas de estado finito.



Identificamos los estados:

- PAR
- IMPAR

Identificamos la entrada:

- Una variable "char"

Ejemplo 1

- Realizar un programa que permita ingresar por teclado cualquier secuencia de 0 y 1. En pantalla se debe mostrar si la cantidad de 1 ingresados es PAR o IMPAR. Utilizar maquinas de estado finito.



Identificamos los estados:

- PAR
- IMPAR

Identificamos la entrada:

- Una variable "char"

Identificamos las salidas:

- 0 : imprimo PAR
- 1 : imprimo IMPAR

Ejemplo 1

- Realizar un programa que permita ingresar por teclado cualquier secuencia de 0 y 1. En pantalla se debe mostrar si la cantidad de 1 ingresados es PAR o IMPAR. Utilizar maquinas de estado finito.



Identificamos los estados:

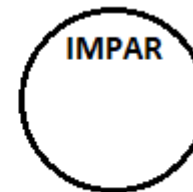
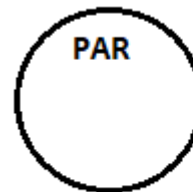
- PAR
- IMPAR

Identificamos la entrada:

- Una variable "char"

Identificamos las salidas:

- 0 : imprimo PAR
- 1 : imprimo IMPAR



Ejemplo 1

- Realizar un programa que permita ingresar por teclado cualquier secuencia de 0 y 1. En pantalla se debe mostrar si la cantidad de 1 ingresados es PAR o IMPAR. Utilizar maquinas de estado finito.



Identificamos los estados:

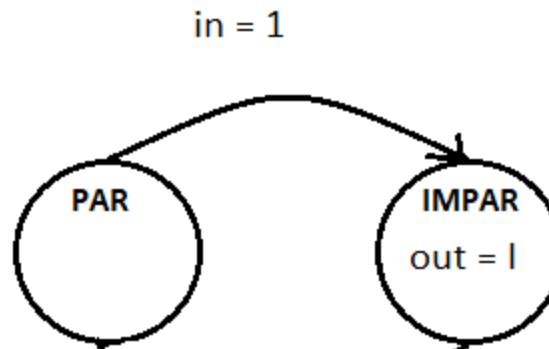
- PAR
- IMPAR

Identificamos la entrada:

- Una variable "char"

Identificamos las salidas:

- 0 : imprimo PAR
- 1 : imprimo IMPAR



Ejemplo 1

- Realizar un programa que permita ingresar por teclado cualquier secuencia de 0 y 1. En pantalla se debe mostrar si la cantidad de 1 ingresados es PAR o IMPAR. Utilizar maquinas de estado finito.



Identificamos los estados:

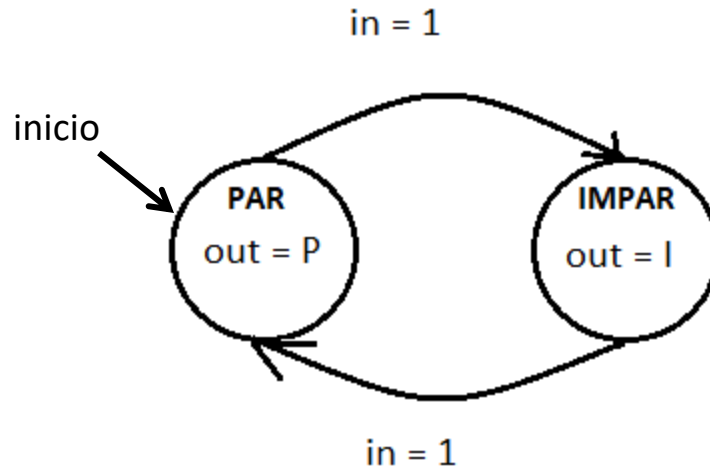
- PAR
- IMPAR

Identificamos la entrada:

- Una variable "char"

Identificamos las salidas:

- 0 : imprimo PAR
- 1 : imprimo IMPAR



¿Moore o Mealy?

Ejemplo 2

- Realizar un programa que detecte la secuencia de caracteres “abcd” ingresadas por teclado. Utilizar máquinas de estado.

¿Moore o Mealy?

Ejemplo 3

- Tomamos la MISMA máquina del Ejemplo 2 (4 estados $A \rightarrow B \rightarrow C \rightarrow D$ que detectan abcd).
- Ahora además queremos contar CUÁNTAS veces apareció abcd en la entrada. A la TERCERA vez que se detecta imprimir en pantalla “ALARMA ACTIVA” y volver a contar desde cero.
- Para contar, acumular o guardar $\rightarrow$ agregamos VARIABLES.
- Los mismos estados + memoria = EFSM (extended FSM).

¿Dónde van las variables en el diagrama?

- Formato: **evento** [**guarda**] / **acción**
- LEER una variable:
 - va en la **GUARDA** -> **entrada** [**contador == 3**] / **acción**
- ESCRIBIR/actualizar variables:
 - estilo Mealy -> en la TRANSICIÓN, tras la barra: **entrada** / **contador++**
 - estilo Moore -> en el ESTADO
 - Podemos usar ambos estilos en una misma maquina, el que nos convenga.

Puede no haber acción: **evento** [**guarda**]

Puede no haber evento: [**guarda**] / **acción**

Puede no haber guarda: **evento** / **acción**

Y otras combinaciones. Lo importante: se escribe solo lo que necesitamos poner en evidencia

Ejemplo 3

- Realizar un programa que detecte la secuencia de caracteres “abcd” ingresadas por teclado. Contar CUÁNTAS veces apareció abcd en la entrada. A la TERCERA vez que se detecta imprimir en pantalla “ALARMA ACTIVA” y volver a contar desde cero.

Ejemplo 4

(<https://sergioprado.org/maquina-de-estados-em-c/>)

- Mostrar en pantalla los datos que ingresan con el siguiente protocolo:

|STX|CANT_DATOS|DATOS|CHK|ETX|

STX (1 Byte) -> Inicio (0x02)

CANT_DATOS (1 Byte) -> Cantidad de datos

DATOS (N Bytes) -> Datos

CHK (1 Byte) -> Checksum

ETX (1 Byte) -> Fin (0x03)

Nota:

El checksum es el resultado de la operatoria XOR de todos los datos.

The image features a dark blue background. A vertical stripe of a lighter blue shade runs along the left side. A large, semi-transparent blue circle is positioned on the right side of the frame. The word "FIN" is written in white, bold, uppercase letters, centered within the circle.

FIN